

Object-Oriented Programming (Classes and Objects)

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1. Methods and Constructors.....	1
2. Method and Constructor Overloading	3
3. Differences Between Methods and Constructors.....	5
4. Summary of Key Concepts.....	6
5. References.....	6

5.2.1 Methods and Constructors

✓ 1. Methods in Java

A method is a block of code that performs a specific task. Methods are used to define the behavior of an object and promote code reusability.

◆ Defining a Method

```
return_type methodName(parameter_list) {  
    // method body  
}
```

✓ Example:

```
public int add(int a, int b) {  
    return a + b;  
}
```

◆ Calling a Method

Methods are called using object references or directly if static:

```
MyClass obj = new MyClass();  
obj.add(10, 20); // instance method  
MyClass.staticMethod(); // static method
```

◆ Properties of a Class in Java:

1. Has a return type (e.g., int, void, String, etc.).
2. Must have a name, which can be different from the class name.
3. Can be called multiple times using object reference or class name (if static).
4. Can be overloaded (same name, different parameters).
5. Can have access modifiers (public, private, etc.).
6. Supports recursion (a method calling itself).
7. May be static, abstract, final, or synchronized.
8. Defined inside a class.
9. Improves code reusability and modularity.

◆ Types of Methods

- **Instance methods** – require an object to be called.
- **Static methods** – can be called using the class name.
- **Abstract methods** – declared without a body; defined in subclasses.
- **Final methods** – cannot be overridden.
- **Synchronized methods** – used in multithreading to avoid thread interference.

✓ 2. Constructors in Java

A constructor is a special method used to initialize objects. It has the same name as the class and no return type.

◆ Syntax of Constructor

```
class MyClass {  
    MyClass() {  
        // constructor body  
    }  
}
```

◆ 1. Properties of an Object in Java:

1. Same name as the class in which it is defined.
2. No return type, not even void.
3. Automatically invoked when an object is created using new.
4. Used to initialize instance variables of a class.
5. Can be overloaded (constructor overloading).
6. Cannot be inherited, but a subclass constructor can call a superclass constructor using super().
7. Java provides a default constructor if none is defined.
8. May or may not accept parameters (default or parameterized).
9. Cannot be abstract, final, static, or synchronized.
10. Defined inside a class like methods.

◆ Types of Constructors

1. **Default Constructor** – Provided by Java if no constructor is defined.

```
MyClass obj = new MyClass(); // calls default constructor
```

2. **Parameterized Constructor** – Accepts arguments to initialize fields.

```
MyClass(int x, String y) { ... }
```

5.2.2 Method and Constructor Overloading

◆ What is Method Overloading?

Method Overloading is a feature in Java that allows a class to have **more than one method with the same name**, but with **different parameter lists** (type, number, or order of parameters). It is a type of **compile-time polymorphism** (also called **static polymorphism**).

◆ Rules for Method Overloading

- Methods must have **the same name**.
- Methods must have **different parameter lists**:
 - Different number of parameters, **or**
 - Different types of parameters, **or**
 - Different order of parameters (if types differ)
- **Return type can be the same or different**, but return type **alone cannot be used** to distinguish overloaded methods.

◆ Example of Method Overloading

```
public class Calculator {  
  
    // Method 1  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2 - Different number of parameters  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method 3 - Different type of parameters  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    // Method 4 - Different order of parameters  
    int add(int a, double b) {  
        return a + (int)b;  
    }  
}
```

◆ Why Use Method Overloading?

- Increases **readability and clarity** of the code.
- Promotes **code reusability**.
- Allows the same operation to be performed in **different ways** depending on input.

◆ What is Constructor Overloading?

Constructor Overloading in Java means having **more than one constructor** in the same class, each with a **different parameter list** (different type, number, or order of parameters). It allows **multiple ways to initialize objects** of a class.

◆ Key Rules for Constructor Overloading

- Constructors must have the **same name as the class**.
- Each constructor must have a **different parameter list**:
 - Different number of parameters
 - Different types of parameters
 - Different order of parameters
- **Return type is not required**, not even void.

◆ Example of Constructor Overloading

```
public class Student {  
    String name;  
    int age;  
  
    // Constructor 1: No parameters  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Constructor 2: One parameter  
    Student(String n) {  
        name = n;  
        age = 0;  
    }  
  
    // Constructor 3: Two parameters  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
}
```

◆ Usage Example

```
Student s1 = new Student();           // Calls Constructor 1  
Student s2 = new Student("Alice");    // Calls Constructor 2  
Student s3 = new Student("Bob", 20);  // Calls Constructor 3
```

◆ Why Use Constructor Overloading?

- To provide **multiple ways of initializing** an object.
- To handle **different types of input data** while creating an object.
- To **simplify object creation** for different scenarios.

◆ Calling One Constructor from Another (Constructor Chaining)

You can use the `this()` keyword to call one constructor from another within the same class.

```
public class Student {
    String name;
    int age;

    Student() {
        this("Unknown", 0); // Calls another constructor
    }

    Student(String n, int a) {
        name = n;
        age = a;
    }
}
```

5.2.3 Differences Between Methods and Constructors

Feature	Constructor	Method
Purpose	Initializes objects	Performs operations
Name	Same as class	Any valid identifier
Return Type	No return type (not even void)	Must have a return type
Called Automatically	Yes, during object creation	No, must be called explicitly
Inheritance	Not inherited	Inherited by subclasses
Overloading	Allowed	Allowed
Invocation	Called with new keyword	Called using dot operator

➤ Summary of Key Concepts:

- **Methods** define reusable blocks of code that perform specific tasks and can return values. They improve code modularity.
- **Constructors** are special methods used to initialize objects. They have the same name as the class and no return type.
- **Method Overloading** allows multiple methods with the same name but different parameter lists.
- **Constructor Overloading** enables initializing objects in different ways using constructors with varying parameters.
- The **key difference**: Methods can return values and be called anytime; constructors are automatically invoked during object creation.

➤ References:

- Oracle Java Documentation – <https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html>
- GeeksforGeeks – Methods in Java – <https://www.geeksforgeeks.org/methods-in-java/>
- JavaTpoint – Constructor Overloading – <https://www.javatpoint.com/constructor-overloading-in-java>
- TutorialsPoint – Java Constructors – https://www.tutorialspoint.com/java/java_constructors.htm

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>